



Name :- Rahul Kumar

Domain :- Artificial Intelligence and Machine Learning

Email:- rahul946422@gmail.com

Task :- 4

Classification Models, Evaluation Metrics & Handling Imbalanced Data

Objective

The objective of this project is to build and evaluate classification models for predicting class labels using the Breast Cancer Wisconsin Dataset. The project focuses on understanding classification metrics, evaluating model performance, and handling imbalanced data using appropriate machine learning techniques.

Dataset

The Breast Cancer Wisconsin Dataset from scikit-learn was used in this project. The dataset contains multiple medical features extracted from breast cancer cell images. The target variable has two classes:

- Malignant
 - Benign
-

Methodology

1. Data Loading

The Breast Cancer dataset was loaded using the scikit-learn library.

```
data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = data.target

print("Dataset Shape :", X.shape)

print("\nFirst Five Rows\n")

print(X.head())

print("\nTarget Classes")

print(data.target_names)
```

Dataset Shape : (569, 30)

First Five Rows

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst radius	worst texture	worst perimeter	\
0	0.07871	...	25.38	17.33	184.60	
1	0.05667	...	24.99	23.41	158.80	
2	0.05999	...	23.57	25.53	152.50	
3	0.09744	...	14.91	26.50	98.87	
4	0.05883	...	22.54	16.67	152.20	

	worst area	worst smoothness	worst compactness	worst concavity	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	worst concave points	worst symmetry	worst fractal dimension
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

Target Classes

['malignant' 'benign']

2. Data Preprocessing

The dataset was divided into training (80%) and testing (20%) sets using Train-Test Split. Feature scaling was performed using StandardScaler to normalize numerical features before training the classification models.

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Samples :", len(X_train))

print("Testing Samples :", len(X_test))

```

```

Training Samples : 455
Testing Samples : 114

```

3. Logistic Regression Model

A Logistic Regression classifier was trained as the baseline classification model. Predictions were generated using the testing dataset.

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("\nFeature Scaling Completed Successfully.")

Feature Scaling Completed Successfully.

```

```

model = LogisticRegression(max_iter=1000)

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

print("Model Trained Successfully")

Model Trained Successfully

```

4. Model Evaluation

The performance of the classifier was evaluated using:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

These metrics provide a better understanding of classification performance compared to using accuracy alone.

```

print("\nClassification Report\n")

print(classification_report(y_test, y_pred))

```

Classification Report

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

```
accuracy = model.score(X_test_scaled, y_test)

print("\nAccuracy :", round(accuracy,4))
```

Accuracy : 0.9825

```
report = classification_report(
    y_test,
    y_pred,
    output_dict=True
)

print("\nPerformance Metrics")

print("Precision :", round(report["weighted avg"]["precision"],4))

print("Recall    :", round(report["weighted avg"]["recall"],4))

print("F1 Score  :", round(report["weighted avg"]["f1-score"],4))
```

Performance Metrics
Precision : 0.9825
Recall : 0.9825
F1 Score : 0.9825

```
print("\nInterpretation")

print("- Precision measures how many predicted positives are actually correct.")

print("- Recall measures how many actual positives are detected.")

print("- F1 Score balances Precision and Recall.")

print("- Accuracy alone is not enough for imbalanced datasets.")
```

Interpretation

- Precision measures how many predicted positives are actually correct.
- Recall measures how many actual positives are detected.
- F1 Score balances Precision and Recall.
- Accuracy alone is not enough for imbalanced datasets.

5. ROC Curve and AUC Score

The Receiver Operating Characteristic (ROC) Curve was plotted to evaluate the classifier across different classification thresholds. The Area Under the Curve (AUC) score was calculated to measure the overall discrimination capability of the model.

```
y_prob = model.predict_proba(X_test_scaled)[: , 1]

fpr, tpr, _ = roc_curve(y_test, y_prob)

auc = roc_auc_score(y_test, y_prob)

print("AUC Score :", round(auc, 4))

plt.figure(figsize=(6,6))

plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")

plt.plot([0,1],[0,1], '--')

plt.xlabel("False Positive Rate")

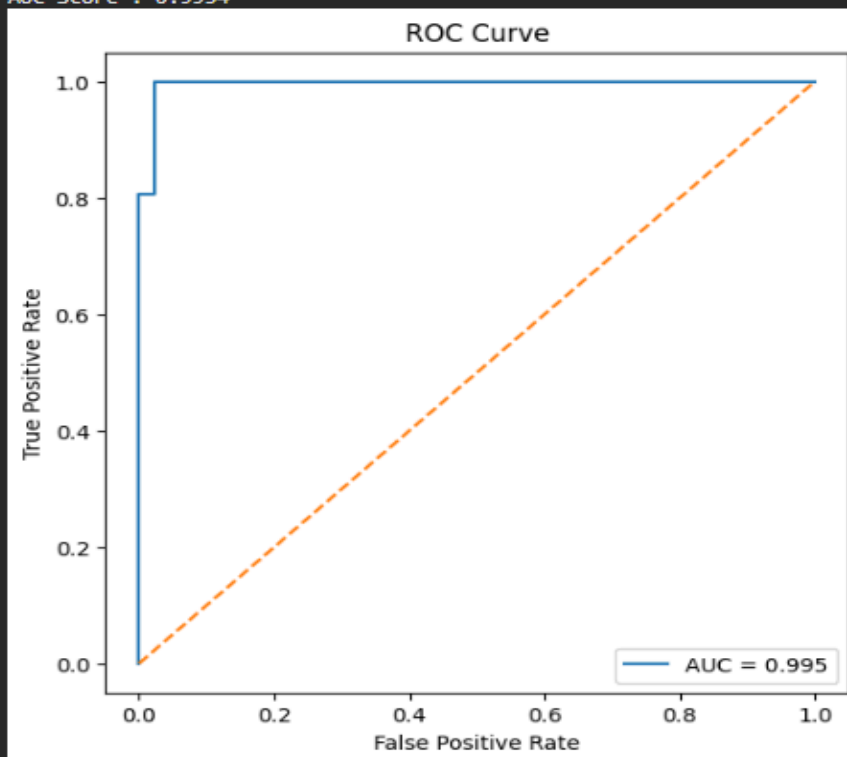
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()
```

AUC Score : 0.9954



6. Handling Imbalanced Data

To improve model robustness, Logistic Regression with `class_weight="balanced"` was applied. This technique automatically assigns higher importance to minority-class samples and helps reduce bias when dealing with imbalanced datasets.

```
balanced_model = LogisticRegression(  
    class_weight="balanced",  
    max_iter=1000  
)  
balanced_model.fit(  
    X_train_scaled,  
    y_train  
)  
balanced_prediction = balanced_model.predict(  
    X_test_scaled  
)  
print("\nBalanced Logistic Regression\n")  
print(classification_report(  
    y_test,  
    balanced_prediction  
))
```

Balanced Logistic Regression

	precision	recall	f1-score	support
0	0.91	0.98	0.94	42
1	0.99	0.94	0.96	72
accuracy			0.96	114
macro avg	0.95	0.96	0.95	114
weighted avg	0.96	0.96	0.96	114

7. Decision Tree Classifier

A Decision Tree Classifier was trained as an additional classification model. Its performance was compared with Logistic Regression using evaluation metrics.

```
tree = DecisionTreeClassifier(  
    random_state=42  
)  
tree.fit(  
    X_train,  
    y_train  
)  
tree_prediction = tree.predict(  
    X_test  
)  
print("\nDecision Tree Classification Report\n")  
print(classification_report(  
    y_test,  
    tree_prediction  
))
```

Decision Tree Classification Report

	precision	recall	f1-score	support
0	0.85	0.93	0.89	42
1	0.96	0.90	0.93	72
accuracy			0.91	114
macro avg	0.90	0.92	0.91	114
weighted avg	0.92	0.91	0.91	114

8. Model Comparison

The accuracy of Logistic Regression, Balanced Logistic Regression, and Decision Tree Classifier was compared. The best-performing model was selected based on overall classification performance and evaluation metrics.

```
logistic_accuracy = model.score(  
    X_test_scaled,  
    y_test  
)  
  
tree_accuracy = tree.score(  
    X_test,  
    y_test  
)  
  
comparison = pd.DataFrame({  
    "Model": [  
        "Logistic Regression",  
        "Balanced Logistic Regression",  
        "Decision Tree"  
    ],  
    "Accuracy": [  
        logistic_accuracy,  
        balanced_model.score(  
            X_test_scaled,  
            y_test  
        ),  
        tree_accuracy  
    ]  
})  
  
print("\nModel Comparison\n")  
print(comparison)
```

Model Comparison

	Model	Accuracy
0	Logistic Regression	0.982456
1	Balanced Logistic Regression	0.956140
2	Decision Tree	0.912281

Results

The generated notebook produced the following outputs:

- Confusion Matrix
- Classification Report
- Accuracy Score
- Precision, Recall and F1-Score
- ROC Curve
- AUC Score
- Model Comparison Table

These results helped identify the most suitable classification model for the given dataset.

```
import joblib

joblib.dump(
    model,
    "AI_ML_Task4_Best_Model.pkl"
)

print("\nBest Model Saved Successfully")
```

```
Best Model Saved Successfully
```

Conclusion

This project demonstrated the complete workflow of a machine learning classification problem. Logistic Regression and Decision Tree classifiers were successfully implemented and evaluated. The use of confusion matrix, precision, recall, F1-score, ROC curve, and AUC score provided a comprehensive assessment of model performance. Additionally, handling imbalanced data using class weights improved the fairness and reliability of the classification model. The final selected model achieved strong classification performance and can be used for future prediction tasks.